

Operációs rendszerek II.

UNIX: folyamatok közötti szinkronizáció, kommunikáció



Várkonyiné Kóczy Annamária

Professzor

Informatika Tanszék

koczya@uj.ssk

varkonyi-koczy@uni-obuda.hu

Felhasznált irodalom:

- Kóczy-Kondorosi (szerk.): Operációs rendszerek mérnöki megközelítésben
- Tanenbaum: Modern Operating Systems 2nd. Ed.
- Silberschatz, Galvin, Gagne: Operating System Concepts



15. Folyamatok közötti szinkronizáció és kommunikáció

- Ismétlés, bevezetés
- Jelzések (események keltése és kezelése)
- Csővezetékek (FIFO kommunikáció)
- Szemaforok
- Üzenetsorok (diszkrét, típusos üzenetek)
- Osztott memória (azonos fizikai memóriaterület használata több folyamatban)
- Hálózati (socket) kommunikáció
- Távoli eljáráshívás UNIX módra

Bevezetés - ismételés

- A kernel
 - felügyeli a folyamatokat, menedzseli az erőforrásokat
 - elkülöníti egymástól a folyamatokat (mindenkinek saját virtuális gép)
 - réteges, moduláris felépítésű
 - a folyamatok a kernellel rendszerhívásokon keresztül kommunikálnak
- A folyamatok
 - a felhasználói programok futás alatt álló példányai
 - UNIX alatt szülő-gyerek kapcsolatban állnak egymással

Bevezetés - ismételés

- Szinkronizáció
 - Egy folyamat végrehajtásának időbeni korlátozása egy másik folyamat futásától vagy külső eseménytől függően
 - A UNIX rendszerekben a legegyszerűbb eszköze a jelzés (signal)
- Kommunikáció
 - A folyamatok együttműködéséhez információcserére van szükség
 - közös memórián keresztül, üzenetekkel, távoli eljárashívás (RPC)
 - erőforrások védelme, kritikus szakasz, szemafor
 - blokkoló (szinkron), nem blokkoló (aszinkron)

A kommunikáció (alap)csatornái

- Jelzések
 - események keltése és kezelése
- Csővezetékek
 - FIFO kommunikáció
- Szemaforok
- Üzenetsorok
 - diszkrét, típusos üzenetek folyamatok között
- Osztott memória
 - azonos fizikai memóriaterület használata több folyamatban
- Hálózati (socket) kommunikáció
- Távoli eljáráshívás UNIX módra

UNIX jelzések

- Cél: egy folyamat (vagy folyamatcsoport)
 - értesítése a kernelben, más folyamatokban, valamint önmagában bekövetkezett eseményekről
 - szinkronizálása más folyamatokhoz (ma már van jobb megoldás)
- Jelzés típusa (SIGINT, SIGCHLD, SIGKILL, ...)
 - rendszer: kivételek (pl. hibák), kvóta, riasztás, értesítés (egy gyerek leállt)
 - felhasználói: emberek (ctrl + c, ctrl + z), folyamatok (tetszőleges céllal)

UNIX jelzések (folyt.)

- A működés áttekintése
 - jelzés keltése (rendszerhívás vagy valamilyen esemény bekövetkezése)
 - a kernel értesíti a címzetteket a jelzésről
 - a címzett egy jelzéskezelő eljárásában fogadja a jelzést
- Problémák a megvalósítással
 - a keltés és a kézbesítés időben szétválik (akár elég messze is)
 - sokféle implementáció, némelyik nem túl megbízható (elveszhet jelzés), pl. korai rendszerek (System V)
 - Kompatibilitási problémák (System V. ↔ BSD),
 - POSIX.1 szabvány (IEEE 90) → szabványos interfész definiálása
 - Később részletesen tárgyaljuk

Jelzések keltése és kezelése

- Jelzések keltése (sok minden kiválthat jelzést ...)
 - A futó folyamat
 - Más folyamat
 - OS
 - Külső esemény
 - IT
- Fő kategóriák
 - Kivételek (exception)
 - Más folyamatok
 - Terminál megszakítások
 - Munkamenet kezelés
 - Kvóta jelzések
 - Értesítések
 - Riasztás

Jelzések keltése és kezelése

- Jelzések kezelése

- Akkor számít kézbesítettnek, ha a címzett folyamat veszi és csinál valamit
- többféle kezelési eljárás lehetséges, bizonyos keretek között állítható. 5 féle alapértelmezett (default) akció van
 - Megszakítás (abortálás). Core: core dump és leállítás (`exit()`)
 - Kilépés. Term: leállítás (`exit()`)
 - Figyelmen kívül hagyás. Ign: figyelmen kívül hagyás
 - Felfüggesztés. Stop: felfüggesztés
 - Folytatás. Cont: visszatérés a felfüggesztett állapotból (vagy ignore)
- az alkalmazás saját kezelőfüggvényét is megadhat

```
signal(SIGALRM, alarm);    /* kezelőfüggvény beállítása */
void alarm(int signum) { ... } /* a kezelőfüggvény */
```
- a jelzés típusától függ, hogy a fentiek közül mi az alapértelmezett, illetve minek a beállítása engedélyezett
 - pl. a SIGKILL és SIGSTOP nem hagyható figyelmen kívül és nem definiálható rá jelzéskezelő függvény

Jelzések keltése és kezelése

- Jelzéshez tartozó akció végrehajtása a címzett folyamat környezetében történik (csak ha az fut), lényeges késleltetés lehet
- A fogadó folyamat tudomására akkor jut egy jelzés érkezése, ha a kernel meghívja az `issig()` rendszerhívást, amikor
 - `userfut`-ba tér vissza rendszerhívásból vagy küldő IT-ből
 - Egy megszakítható eseményre történő várakozás előtt
 - Közvetlenül egy nem megszakítható eseményre történő várakozás után
- Ha az `issig()` IGAZ-zal tér vissza, akkor
 - a kernel kezeli a jelzést a `psig()` hívás segítségével
 - Végrehajtja az alapértelmezett akciót vagy
 - `sendsig()`-gel meghívja a folyamatkezelő függvényt, de előtte a folyamat kilép a `kernelfut` módból
- a `psig()` megoldja, hogy a jelzéskezelés után a folyamat normálisan folytatódjék (verem és környezet mentés, ...) 10

UNIX jelzések: példák

- System V.:15, ma ált. 31 féle jelzés
- Néhány tipikus példa:

SIGABRT	folyamat megszakítása	megszakítás
SIGTERM	folyamat leállítás	foly. leállítása
SIGSEGV	szegmentációs hiba	megszakítás
SIGALRM	valósidejű órariasztás	kilépés

```
signal(SIGALRM, alarm);    /* jelzéskezelő függvény beállítása */  
void alarm(int sigum) { ... }    /* az eljárás */  
alarm(30);                /* alkalmazás: ALARM jelzés 30mp múlva */
```

SIGBUS	buszhiba	megszakítás
SIGPIPE	olvasó nélküli csővezetékbe írás	kilépés

Megbízhatatlan jelzések

- Az eredeti System V. jelzéskezelő rendszere megbízhatatlan
 - A kernel a jelzés kezelésekor minden alkalommal visszaállítja az alapértelmezett akciót, így a folyamat jelzéskezelő eljárásának gondoskodnia kell az egyedi kezelés ismételt beállításáról. Ez versenyhelyzetet teremt a jelzések keletkezése és az egyedi akciók beállítása között
 - A jelzéseket táblát a kernel a folyamathoz kötött u-területen tárolja, amihez csak az aktuális folyamatnál fér hozzá
 - Hiányzott a jelzések kézbesítésének átmeneti blokkolása, illetve az olyan munkamenetek kezelése, ahol folyamatok csoportosan kezelhetők a terminál-hozzáférés során.

Megbízható jelzések

- A BSD 4.2 új jelzésrendszert vezetett be (AT&T SVR3 is)
- POSIX.1 szabvány
- Közös szolgáltatások
 - Perzisztens kezelők (persistent handlers)
 - Maszkolás (masking)
 - Alvó folyamatok (sleeping processes).
 - Felszabadítás és várakozás (unblock and wait)
- Az SVR3 implementáció
 - egyszerre nem képes több jelzést blokkolni
 - hiányzik a munkafolyamat-kezelés

BSD jelzésmenedzsment

- A 4.2BSD volt az első megbízható jelzésrendszer
- `sigsetmask()`: egy operáció több jelzésen is végrehajtható
- A kernel gondoskodik a maszkolt jelzések helyes és automatikus beállításáról
- A jelzésekhez különálló vermet használ, így tudja kezelni például a verem túlcsoordulását jelző jelzést is
- Új jelzések a munkafolyamat-kezelés területén
- képes a jelzések által félbeszakított, hosszú ideig futó rendszerhívások automatikus újrahívására abortálódás helyett
- A BSD jelzésinterfész hatékony és flexibilis, de nem kompatibilis az eredeti AT&T interfésszel.

Az SVR4 jelzések

- Közös interfészek, amelyek mindkét (BSD, SysV) UNIX-variánssal használhatóak: POSIX.1 jelzésinterfész
- SVR4: a BSD és korábbi SVR3 variánsok és a nem megbízható jelzések teljes halmazát lefedi
- A kernel a folyamatok felébresztése nélkül képes ellenőrizni, hogy azok foglalkoznak-e a jelzéssel (vagy default akció kell)
 - 1 jelzésből max. 1 lehet kezeletlen
 - Ha a folyamat megszakítható várakozásban van és a jelzés nem blokkolt, akkor a kernel felébreszti a folyamatot.

Kivételkezelés

- Kivétel: valamilyen nem megszokott szituáció, tipikusan valamilyen hiba, pl. hibás címezés, nullával való osztás stb.
- Hibamegszakítást okoz
- UNIX problémák:
- a kivételkezelés a kivétellel megegyező kontextusban fut, így nem tud hozzáférni a teljes regiszter tartalmakhoz
- a jelzéseket alapvetően egyszálú folyamatok számára találták ki; többszálú végrehajtást is támogató UNIX-variánsok (például Solaris 7) csak nehezen tudják alkalmazni a jelzéseket.

UNIX csővezetékek: `pipe()`

- Cél: folyamatok közötti adatátvitel
- Jellemzők
 - csak szülő-gyerek (leszármazott, testvér) viszonylatban
 - adatfolyam (nincs üzenethatár, tipizálás)
 - egyirányú adatfolyam (író → olvasó) (több író és olvasó is lehet!)
 - limitált kapacitás: pl. 4k (Linux < 2.6.11), 65k (Linux ≥ 2.6.11)
- A megvalósítás
 - egy folyamat létrehoz egy csővezeték (`pipe()`)
 - a kernel létrehozza az adatstruktúrákat és olvasási/írási leírókat ad vissza (a folyamat továbbadja a leírókat a gyerekeinek)
 - a leírók segítségével kommunikálnak a folyamatok (`read()`, `write()`)
- Korlátok, problémák
 - nincs címezés, tipizálás, üzenethatár
 - csak a „rokonságban” működik

UNIX elnevezett csővezetékek (named pipe, FIFO)

- Az egyszerű csővezetékek legkomolyabb problémájának megoldása
 - független folyamatok kommunikációja
 - avagy egy már létező csővezeték elérése egy másik folyamat által
- Jellemzők
 - nem csak szülő-gyerek viszonylatban alkalmazható
 - ugyanúgy működik, mint a csővezeték
 - a létrehozás a fájlrendszer segítségével történik(`mknod( )`)
 - A folyamatok halála után is megmarad az adat benne
 - Kevésbé biztonságos
 - A törlése elfelejtődhet
 - lehetséges a kétirányú kommunikáció (megnyitás olvasásra és írásra) (Kétirányú csővezeték, SVR4)

Folyamat nyomkövetés: ptrace ()

- Egy (szülő) folyamat befolyásolhatja a gyermeke futását
 - adat olvasása és írása a gyermek címtartományában
 - adat olvasása és írása a gyermek u-területén
 - a gyermek folyamat általános célú regisztereinek írása és olvasása
 - adott jelzések „elkapása”, aminek során a kernel a gyermeket felfüggeszti és értesíti a szülőt a jelzésről
 - megfigyelési pontok (watchpoints) beállítása és törlése a gyermek címtartományában
 - a leállított gyermek folyamat futásának folytatása
 - a gyermek futásának folytatása egy utasítás végrehajtására
 - a gyermek futásának leállítása
 - a gyermek engedélyezheti a szülő számára a nyomkövetést.
- Korlátok:
 - Csak a saját gyereket követheti nyomon
 - Nagyon sok környezetváltást igényel, így nem hatékony
 - Már futó folyamatnál nem működik, mert a gyermeknek előbb engedélyeznie kell a nyomkövetést
 - setuid jelzéssel ellátott programok nyomkövetése biztonsági okokból tiltott

UNIX System V IPC

- Cél: folyamatok közötti „szabványos”, egységes kommunikáció
 - adatátvitel
 - szinkronizáció
- Közös alapok (fogalmak)
 - erőforrás: a kommunikáció eszköze (l. lentebb)
 - kulcs: azonosító az erőforrás eléréséhez (egy 32 bites szám)
 - közös kezelőfüggvények: `*ctl()`, `*get( ... kulcs ...)`
 - jogosultsági rendszer:
 - létrehozó, tulajdonos és csoportjaik
 - a szokásos hozzáférés-szabályozási rendszer (felhasználó, csoport, mások)
- Erőforrások
 - szemaforok
 - üzenetsorok
 - osztott memória

UNIX System V IPC: szemaforok

- Cél: folyamatok közötti szinkronizáció
 - P() és V() operátorok
 - szemaforcsoportok kezelése
- Használat
 - `sem_id = semget(kulcs, szám, opciók);`
 - adott számú szemaforhoz nyújt hozzáférést (adott kulccsal és opciókkal), (a tényleges létrehozás és az egyszerű hivatkozás az opciókban válik szét)
 - az ops struktúrában leírt műveletek végrehajtása :
`status = semop(sem_id, ops, ops_méret);`
 - egyszerre több művelet, több szemaforon is végrehajtható
 - blokkoló és nem blokkoló P() operáció is lehetséges
 - egyszerű tranzakció kezelésre is van lehetőség
 - A sem létrehozás és inicializálás nem atomi művelet!

UNIX System V IPC: üzenetsorok

- Cél: folyamatok közötti adatátvitel
 - diszkrét, tipizált üzenetek láncolt listában
 - nincs címezés, üzenetszórás
- Használat
 - `msgq_id = msgget(kulcs, opciók);`
 - adott kulcsú üzenetsorhoz nyújt hozzáférést (adott opciókkal)
 - (a tényleges létrehozás és az egyszerű hivatkozás az opciókban válik szét)
 - üzenetküldés (az msg tartalmaz egy típus azonosítót is):
`msgsnd(msgq_id, msg, méret, opciók);`
 - vétel: `msgrcv(msgq_id, msg, méret, típus, opciók);`
 - a típus (egész szám) beállításával szűrést valósíthatunk meg
 - = 0 a következő üzenet (tetszőleges típusú)
 - > 0 a következő adott típusú üzenet
 - < 0 a következő üzenet, amelynek a típusa kisebb vagy egyenlő

UNIX System V IPC: osztott memória

- Cél: folyamatok közötti egyszerű és gyors adatátvitel
 - a kernel helyett közvetlen adatátviteli csatorna (osztott memória régió)
 - a fizikai memória elkülönített része, amely közösen használható

- Használat

```
shm_id = shmget(kulcs, méret, opciók);
```

- adott kulcsú osztott memóriához nyújt hozzáférést (adott opciókkal)
(a tényleges létrehozás és az egyszerű hivatkozás az opciókban válik szét)
- hozzárendelés saját virtuális címtartományhoz:

```
változó = (típus) shmat(...);
```


az adott változót hozzákötjük a visszkapott címhez
- lecsatolás: `shmdt(cím);`
- a kölcsönös kizárást meg kell valósítani (pl. szemaforokkal)

UNIX hálózati (socket) kommunikáció

- Cél: címezéssel és protokollokkal támogatott adatátvitel
 - Soros vonali terminálillesztés továbbfejlődése
 - Pont-pont adatkapcsolat
 - tetszőleges folyamatok között kliens – szerver architektúrában
 - többféle célra (folyamatok közötti és gépek közötti hálózati komm.)
 - sokféle protokollt támogat
 - többféle címezés
 - UNIX: a TCP/IP protokollcsalád alapján
- Fogalmak
 - Socket: a kommunikációs csatorna leírója, absztrakt objektum
 - hálózati csatoló avagy azonosító: a kommunikáció végpontja
 - IP cím és portszám

UNIX „hálózati” (socket) kommunikáció

- 2 típus:
 - Folyam (stream) (TCP)
 - megbízható, sorrendhelyes, kétirányú kapcsolat
 - Üzenetszórás (datagram) (UDP)
 - kapcsolatmentes csomagküldés, a sorrend és a megérkezés nem garantált
- A socket hívások a hívó folyamat környezetében hajtódnak végre
- A hibák szinkron módon jelentkeznek a hívónál is

UNIX „hálózati” (socket) kommunikáció

- Használat

- létrehozás `sfd = socket(domén, típus, protokoll);`
- kötés (a lokális gép egy portjához, szerver alkalmazás)
`szerver: bind(sfd, cím, ...);`
- adatút felépítése (a másik gép egy portjáig, kliens alkalmazás)
`kliens: connect(sfd, cím, ...);`
- konfigurálás (várakozási sor hossza)
`szerver: listen(sfd, sor_mérete);`
- kérés fogadása (a sorban következő)
`szerver: accept(sfd, cím, ...);`
- válasz(adat) küldése/fogadása
`send(sfd, üzenet, ...);`
`recv(sfd, üzenet, ...);`

- Lezárás

`shutdown(sfd);`

(Sun) RPC (távoli eljáráshívás)

- A socket kommunikációra épülő „elosztott rendszer” infrastruktúra
- Cél:
 - folyamatok közötti magas szintű kommunikáció
 - távoli eljárások meghívása (másik folyamatban, akár másik gépen)
 - programozói támogatás: interfész leírás + programgenerálás

(Sun) RPC (távoli eljáráshívás) (folyt.)

- Fogalmak
 - RPC nyelv: a hívható eljárások és típusaik (interfész) leírása
 - azonosítók: a leírásban megadott egyedi számok (program, eljárás)
 - portmapper: a programazonosítók és a hálózati portok összerendelése
 - rpcgen: a leírásból C programkódot generáló program
- A Sun RPC technológia részei
 - interfész leírás
 - programgenerátor
 - kommunikációs infrastruktúra

Összefoglalás

- Klasszikus kommunikációs formák:
 - Jelzések (signal)
 - Csővezetékek (pipe)
 - FIFO (elnevezett csővezeték, named pipe)
- System V IPC
 - Szemaforok
 - Üzenetsorok
 - Osztott memória
- (nem csak) Hálózati kommunikáció
 - hálózati csatoló (socket) kommunikáció
 - Sun RPC – távoli eljáráshívás UNIX módra